Name	Vedant Dinkar Patil
UID no.	2024300170
Experiment No.	9

AIM:	To design and implement a Fuzzy Logic Controller (FLC)
PROBLEM STATEMENT :	To design and implement a Fuzzy Logic Controller (FLC) that dynamically adjusts the green light duration at an intersection based on traffic density and waiting time, ensuring smoother traffic flow and reduced congestion.
Theory:	A Fuzzy Logic Controller (FLC) is a control system based on fuzzy set theory that models human reasoning and decision-making using linguistic rules rather than precise equations. Unlike classical binary logic, fuzzy logic uses membership functions to handle degrees of truth ranging from 0 to 1. The core operational steps of an FLC include: • **Fuzzification:** Converting crisp input sensor values (such as traffic density and waiting time) into fuzzy membership degrees using functions like trapezoidal (trapmf) or triangular (trimf) shapes. • **Inference Mechanism:** Applying "If-Then" rules and combining fuzzy inputs using logical operators (where AND evaluates to the minimum value). • **Aggregation:** Combining the output fuzzy sets of all fired rules into a single fuzzy set. • **Defuzzification:** Converting the aggregated fuzzy output back into a crisp numerical value (e.g., using the Centroid of Gravity method) to get the final output, such as the exact green light duration in seconds.

PROGRAM:	!pip install -q scikit-fuzzy import numpy as np import skfuzzy as fuzz from skfuzzy import control as ctrl import matplotlib.pyplot as plt print("Libraries imported successfully!") traffic_density = ctrl.Antecedent(np.arange(0, 51, 1), 'traffic_density') waiting_time = ctrl.Antecedent(np.arange(0, 121, 1), 'waiting_time') green_duration = ctrl.Consequent(np.arange(0, 91, 1), 'green_duration') print("Input and Output variables defined.") traffic_density['low'] = fuzz.trapmf(traffic_density.universe, [0, 0, 5, 15]) traffic_density['medium'] = fuzz.trimf(traffic_density.universe, [10, 20, 30]) traffic_density['high'] = fuzz.trapmf(traffic_density.universe, [25, 35, 50, 50]) waiting_time['short'] = fuzz.trapmf(waiting_time.universe, [0, 0, 10, 30]) waiting_time['moderate'] = fuzz.trimf(waiting_time.universe, [20, 50, 80]) waiting_time['long'] = fuzz.trapmf(waiting_time.universe, [70, 95, 120, 120]) green_duration['short'] = fuzz.trapmf(green_duration.universe, [0, 0, 10, 30]) green_duration['medium'] = fuzz.trimf(green_duration.universe, [20, 35, 55, 70]) green_duration['long'] = fuzz.trapmf(green_duration.universe, [60, 80, 90, 90]) traffic_density.view() waiting_time.view() green_duration.view() plt.show() density_vals = np.linspace(0, 50, 30) waiting_vals = np.linspace(0, 120, 30) X, Y = np.meshgrid(density_vals, waiting_vals)

```
Z = np.zeros_like(X)

rule1 = ctrl.Rule(traffic_density['low'] & waiting_time['short'],
green_duration['short'])
rule2 = ctrl.Rule(traffic_density['low'] & waiting_time['moderate'],
green_duration['short'])
rule3 = ctrl.Rule(traffic_density['low'] & waiting_time['long'],
green_duration['medium'])

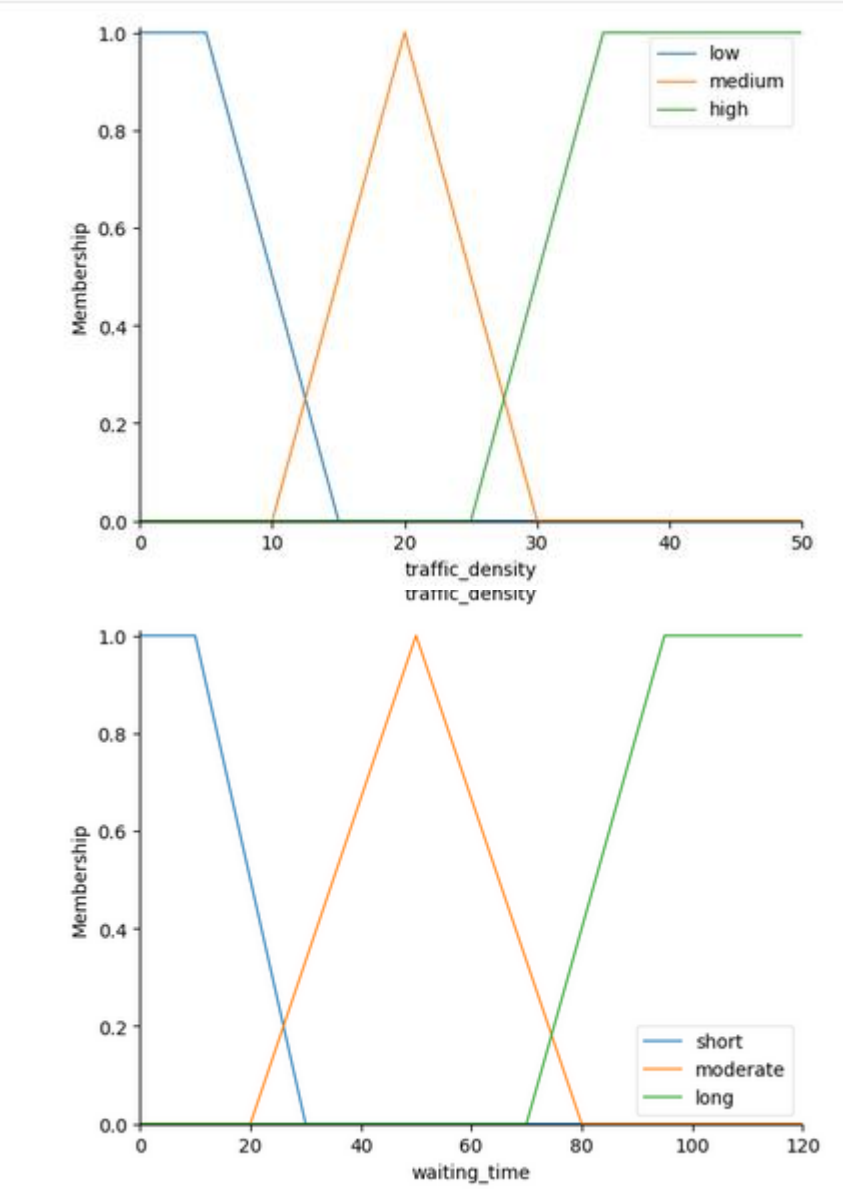
rule4 = ctrl.Rule(traffic_density['medium'] & waiting_time['short'],
green_duration['short'])
rule5 = ctrl.Rule(traffic_density['medium'] & waiting_time['moderate'],
green_duration['medium'])
rule6 = ctrl.Rule(traffic_density['medium'] & waiting_time['long'],
green_duration['long'])

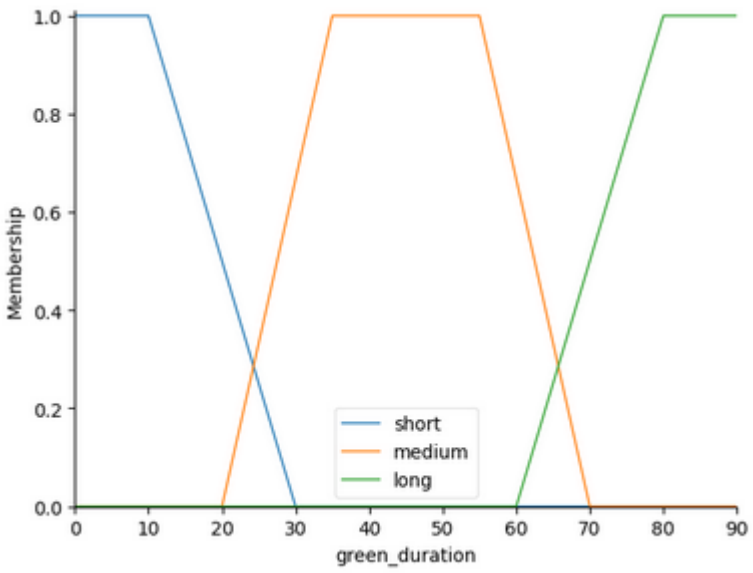
rule7 = ctrl.Rule(traffic_density['high'] & waiting_time['short'],
green_duration['medium'])
rule8 = ctrl.Rule(traffic_density['high'] & waiting_time['moderate'],
green_duration['long'])
rule9 = ctrl.Rule(traffic_density['high'] & waiting_time['long'],
green_duration['long'])

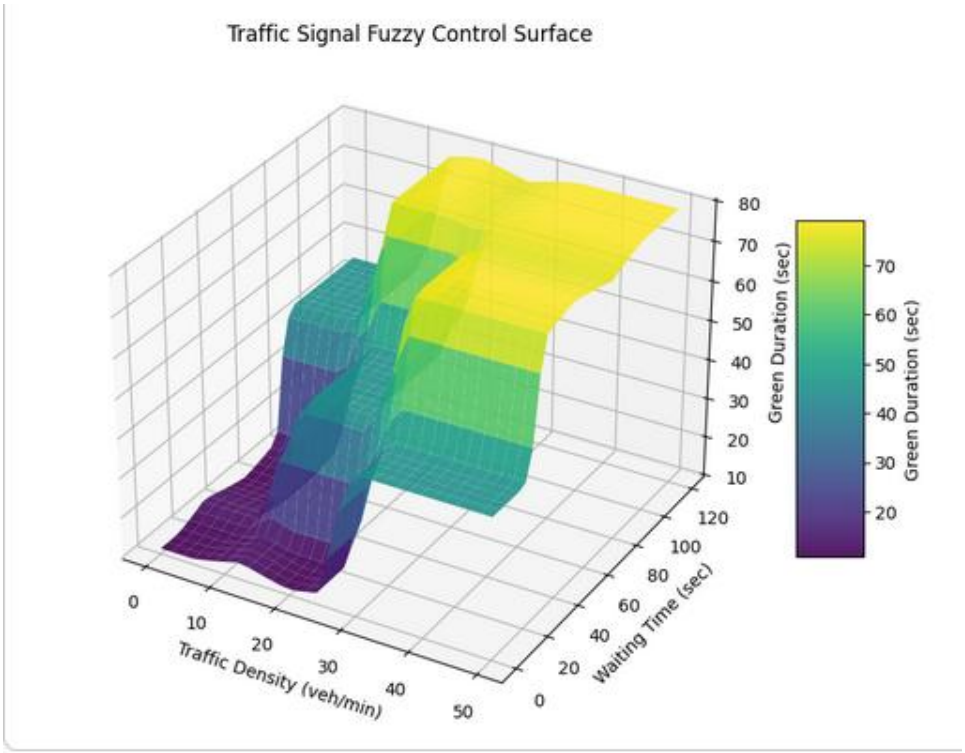
traffic_ctrl = ctrl.ControlSystem([rule1, rule2, rule3, rule4, rule5, rule6,
rule7, rule8, rule9])
traffic_sim = ctrl.ControlSystemSimulation(traffic_ctrl)

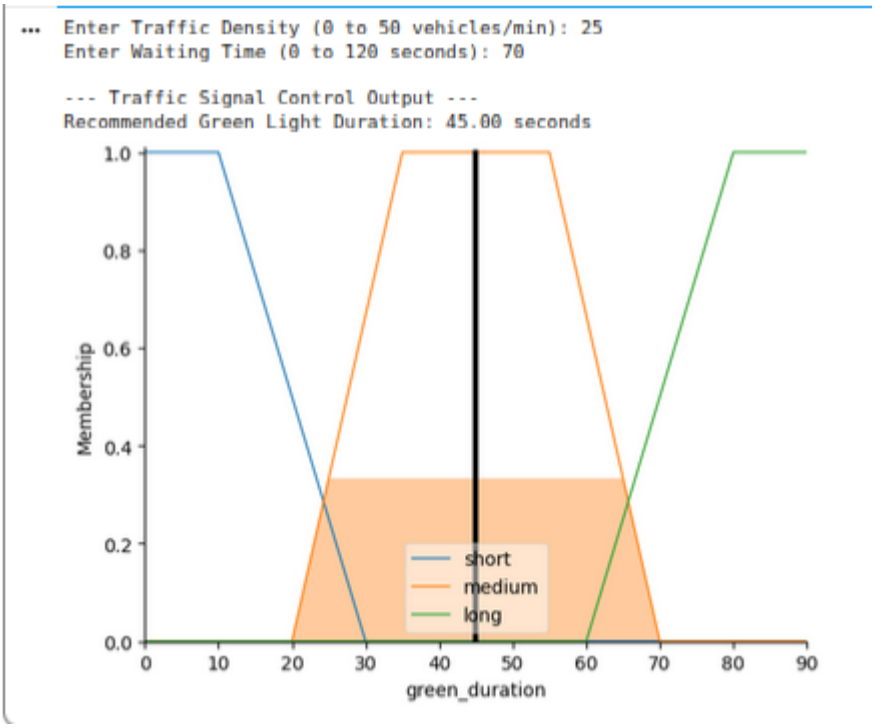
for i in range(len(waiting_vals)):
    for j in range(len(density_vals)):
        try:
            traffic_sim.input['traffic_density'] = X[i, j]
            traffic_sim.input['waiting_time'] = Y[i, j]
            traffic_sim.compute()
            Z[i, j] = traffic_sim.output['green_duration']
        except ValueError:
            Z[i, j] = 0

fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(projection='3d')
surf = ax.plot_surface(X, Y, Z, cmap='viridis', edgecolor='none', alpha=0.9)
```

	ax.set_title('Traffic Signal Fuzzy Control Surface') ax.set_xlabel('Traffic Density (veh/min)') ax.set_ylabel('Waiting Time (sec)') ax.set_zlabel('Green Duration (sec)') fig.colorbar(surf, shrink=0.5, aspect=5, label='Green Duration (sec)') plt.show()
Output:	


1.0
0.8
0.6
0.4
0.2
0.0
Membership
short
medium
long
0
10
20
30
40
50
60
70
80
90
green_duration


Traffic Signal Fuzzy Control Surface
Traffic Density (veh/min)
Waiting Time (sec)
Green Duration (sec)
Green Duration (sec)

	
CONCLUSION:	The Fuzzy Logic Controller was successfully designed and implemented to automate traffic signal management. By translating human-like linguistic rules into mathematical fuzzy sets and processing them through fuzzification, rule inference, and defuzzification, the system successfully computes precise, adaptive green light durations. Visualizations like the membership plots and the 3D control surface confirm that the controller provides smooth, predictable, and intelligent responses across diverse traffic conditions.